



УНИВЕРЗИТЕТ У НОВОМ САДУ ФАКУЛТЕТ ТЕХНИЧКИХ НАУКА



УНИВЕРЗИТЕТ У НОВОМ САДУ
ФАКУЛТЕТ ТЕХНИЧКИХ НАУКА
НОВИ САД

ИСПИТНИ РАД

Кандидат: Селена Грковић
Број индекса: РА97-2024
Предмет: Оперативни системи
Тема рада: Имплементација једноставног оперативног система
Ментор рада: др Вељко Петровић

Нови Сад, мај, 2020.

SADRŽAJ

1. Uvod.....	1
2. Analiza problema.....	2
3. Arhitektura sistema.....	3
3.1. Bootloader.....	4
3.1. Kernel.....	4
4. Programsko rešenje.....	5
4.1. Bootloader.....	5
4.2. Kernel.....	6
4.2.1. Obrada prekida.....	13
4.2.1.1 IDT.....	13
4.2.1.2 ISR.....	13
4.2.1.3 PIC.....	13
4.2.1.4 IRQ.....	13
4.2.2. Drajveri.....	13
4.2.1. Screen Driver.....	13
4.2.1. Timer.....	13
4.2.1. Tastatura.....	13
4.3. Podrška za skladištenje uređaja.....	13
4.3.1. PCI.....	13
4.3.2. VirtIO Block driver.....	10
4.3.3. Read and write operacije.....	11
4.3.4. Disk sloj.....	11

4.3.5. Storage.....	11
4.3.4. Komandna linija.....	11
5. Problemi tokom razvoja.....	13
6. Pokretanje sistema.....	16
7. Dokumentacija.....	17

1. Увод

Овај пројекат представља имплементацију једноставног оперативног система за x86 архитектуру, развијеног коришћењем NASM асемблерског језика и програмског језика C. Циљ пројекта је упознавање са основама рада оперативних система кроз имплементацију кључних компоненти неопходних за покретање и рад једног једноставног система.

Развој је започет имплементацијом boot сектора који BIOS учитава директно приликом покретања рачунара. Након тога реализован је прелазак из Real Mode режима рада процесора у Protected Mode, чиме је омогућено покретање кернела и даљи развој оперативног система.

У оквиру кернела додато је управљање прекидима, имплементирани су Interrupt Descriptor Table (IDT), Interrupt Service Routine (ISR), ремапирање PIC контролера и обрада хардверских прекида (IRQ). Поред тога, имплементирани су и драјвери за рад са екраном, тастатуром и тајмером.

Након проучавања имплементације VirtIO диска у оперативном систему xv6, развијен је сопствени драјвер који је прилагођен x86 окружењу. Имплементирано је скенирање PCI магистрале, проналажење VirtIO block уређаја, иницијализација VirtIO драјвера, конфигурација virtqueue механизма, као и операције читања и писања на диск.

Развијен је и једноставан storage слој који омогућава чување и учитавање података са виртуелног диска. На тај начин систем има могућност да информације остану сачуване и након поновног покретања.

За комуникацију са системом имплементиран је једноставан командни интерфејс shell преко кога је могуће слати основне команде, тестирати рад драјвера и проверити исправност рада система.

2. Анализа проблема

Развој оперативног система подразумева наилажење на низ проблема, почевши од тренутка укључивања рачунара. Након покретања, без оперативног система, не постоји начин да се управља хардвером, читају програми или комуницира са уређајима.

Први изазов је покретање самог система. BIOS може да прочита само boot сектор са диска, па је потребно реализовати bootloader који ће прочитати кернел у меморију. Након његовог покретања, процесор се налази у Real Mode режиму рада, који поседује бројна ограничења.

То нас доводи до још једног проблема, а то је прелазак из 16-bitnog режима рада (Real Mode) у 32-bitni режим рада (Protected Mode), како би се искористио пун потенцијал процесора.

Следећи проблем је комуникација са хардвером, како би се обезбедили механизми за рад са екраном, тастатуром, тајмером и другим уређајима. Поред тога, неопходно је додати и обраду прекида како би систем реаговао на догађаје које генерише хардвер и на софтверске грешке.

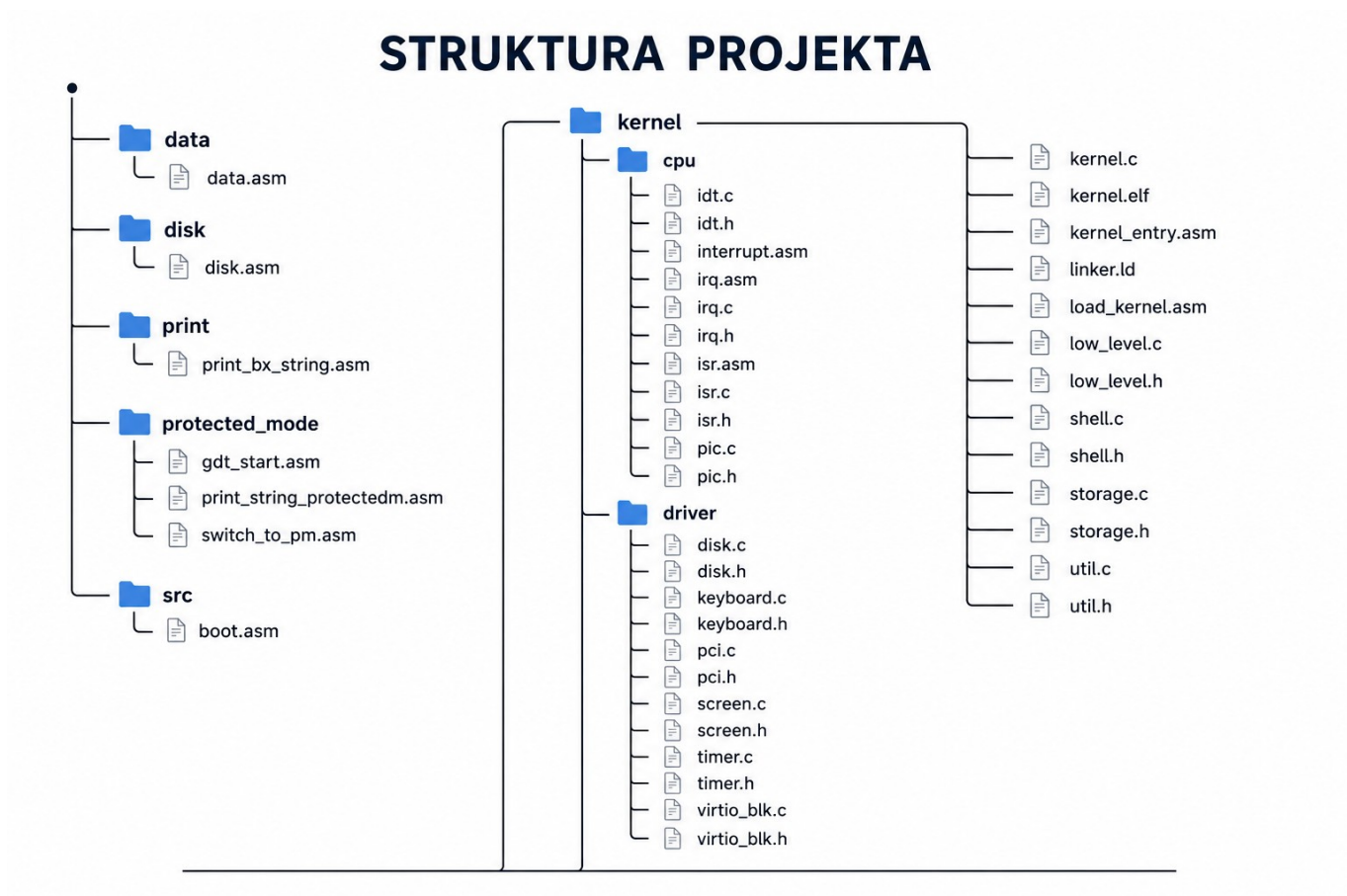
Посебан изазов представљао је приступ диску и трајно чување података, због чега је имплементирана подршка за PCI магистралу и VirtIO Block уређај.

На основу наведених захтева развијен је оперативни систем, а опис решења сваког од проблема биће дат у наставку.

3. Архитектура система

Структура пројекта је подељена на више функционалних целина: bootloader, механизме за прелазак у Protected Mode, кернел, управљање прекидима, драјвере за комуникацију са хардвером и командни интерфејс. Свака целина задужена је за одређени део система и комуницира са осталим целинама преко дефинисаних интерфејса.

Слика 1. структура пројекта



3.1 Bootloader

Bootloader је први део оперативног система који се извршава након покретања рачунара. У њему се иницијализују само основне компоненте система, учитавање кернела са диска и припрема окружења за прелазак у Protected Mode.

Његова имплементација је реализована на следећи начин:

- **src** – датотека boot.asm садржи основну логику bootloader-a и од њега започиње извршавање система .
- **data** – садржи константе и поруке неопходне током процеса покретања.
- **print** – садржи функцију за испис порука на екран током рада у Real Mode режиму.
- **disk** – омогућава читање података са диска и учитавање кернела у меморију.
- **protected_mode** – садржи механизме за прелазак из Real Mode у Protected Mode режим рада.

3.2 Кернел

Након што bootloader учита све податке у меморију и пређе у Protected Mode режим рада, кернел почиње са радом. Он представља централни део оперативног система и задужен је за управљање хардверским ресурсима, обраду прекида, комуникацију са уређајима и интеракцију са корисником.

Функционалности кернела организоване су у више целина:

- **cpu** – садржи компоненте које су задужене за управљање прекидима, као што су IDT табела, ISR и IRQ рутине и PIC контролер.
- **driver** – садржи компоненте за комуникацију са хардверским уређајима, као што су екран, тастатура, тајмер, PCI уређаји и VirtIO диск.
- **shell** – реализује једноставан командни интерфејс преко кога корисник може да комуницира са оперативним системом.
- **storage** – омогућава чување и учитавање података са VirtIO диска.
- **low_level** – садржи функције за приступ процесорским I/O портовима.
- **util** – садржи помоћне функције.

4. Програмско решење

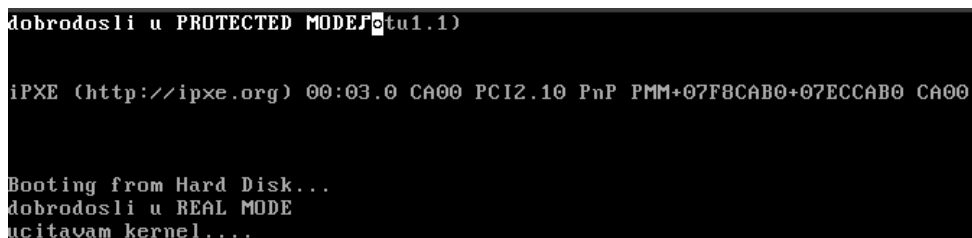
4.1 Bootloader

Пре покретања оперативног система, рачунар ради у веома ограниченом окружењу и нема могућност директног коришћења већине хардверских ресурса. Због тога се приликом укључивања рачунара прво покреће BIOS (Basic Input/Output System), скуп софтверских рутина смештених у меморијском чипу матичне плоче. BIOS иницијализује основне хардверске компоненте и припрема рачунар за учитавање оперативног система.

Након иницијализације хардвера, BIOS претражује први сектор диска, познат као boot сектор. Он заузима првих 512 бајтова диска, а његова последња два бајта морају имати вредност 0xAA55. Ова вредност BIOS-у указује на то да сектор садржи програм за покретање оперативног система.

BIOS учитава boot сектор на меморијску адресу 0x7C00 и предаје му контролу. Од тог тренутка започиње извршавање bootloader-а, реализованог у датотеци boot.asm. Пошто процесор у овом тренутку ради у 16-битном Real Mode режиму рада, у bootloader-у могу да се користе BIOS рутине за основну комуникацију са хардвером. Приступ овим рутинама остварује се путем софтверских прекида. На пример, прекид int 0x13 омогућава приступ диску и учитавање додатних сектора у меморију. Bootloader користи овај прекид како би са диска прочитао кернел оперативног система.

Пре предаје контроле кернелу, потребно је да процесор пређе у 32-битни Protected Mode режим рада. Овај поступак подразумева постављање Глобалне дескрипторске табеле (GDT), омогућавање Protected Mode режима изменом контролних регистара и поновно подешавање сегментних регистара. Након успешног преласка у Protected Mode, bootloader предаје контролу кернелу. Испис успешно извршеног bootloader-а приказан је на слици 1.



```
dobrodosli u PROTECTED MODE (tu1.1)

IPXE (http://ipxe.org) 00:03.0 CA00 PCI2.10 PnP PMM+07F8CAB0+07ECCAB0 CA00

Booting from Hard Disk...
dobrodosli u REAL MODE
ucitavam kernel...
```

4.2 Кернел

Кернел је централни део оперативног система и задужен је за управљање хардвером, обраду прекида и извршавање основних функција оперативног система. Имплементиран је комбинацијом x86 асемблера и програмског језика C.

4.2.1 Обрада прекида

Након преласка у Protected Mode, неопходно је омогућити обраду прекида. Прекиди су сигнали којима хардвер обавештава процесор да је потребно обрадити неки догађај или захтев. У Real Mode режиму прекиди су се обрађивали захваљујући BIOS-у, али након преласка у Protected Mode оперативни систем мора да има сопствени механизам за обраду прекида. У супротном, уколико би се догодио неки прекид, систем би се срушио.

4.2.1.1 IDT

Пошто постоје различите врсте прекида, имплементиран је Interrupt Descriptor Table (IDT). У њој се налазе адресе рутине задужених за обраду процесорских изузетака и хардверских прекида. Без ове табеле процесор не би могао да одреди коју функцију треба позвати приликом настанка прекида.

x86 подржава 256 прекида, а у овом пројекту имплементирано је само 48. Првих 32 су процесорски изузеци, а осталих 16 хардверски прекиди.

4.2.1.2 ISR

Након имплементације IDT табеле било је потребно имплементирати и рутине које ће се извршавати приликом појаве процесорских изузетака (првих 32 из IDT табеле). Због тога су додате ISR (Interrupt Service Routine) рутине, чије су адресе претходно уписане у IDT табелу.

Процесор аутоматски генерише изузетке у ситуацијама као што су дељење нулом или извршавање неважеће инструкције. Приликом настанка изузетка процесор преноси извршавање на одговарајућу ISR рутину. Ако оперативни систем нема рутине за обраду оваквих догађаја, долази до пада система.

Слика 2. пример успешно обрађеног изузетака

```
idt_init();
int a=5;
int b=0;
int c= a/b;
```





Слика 3. пример пада система при дељењу са нулом без имплементираних ISR-а и IDT-а

4.2.1.3 PIC

PIC (Programmable Interrupt Controller) је контролер задужен за прослеђивање прекида које генеришу хардверски уређаји. Процесор не комуницира директно са уређајима као што су тастатура и системски тајмер. Уместо тога, уређаји шаљу захтев за прекид PIC контролеру, који затим прослеђује одговарајући прекид процесору.

Пошто су вектори 0–31 већ резервисани за процесорске изузетке, хардверски прекиди су премештени на другу групу вектора тако што је извршено ремапирање PIC контролера, након чега су смештени у опсег од 32 до 47.

PIC контролер подржава осам IRQ линија. Због касније потребе за већим бројем хардверских прекида, коришћена су два повезана PIC контролера, чиме је обезбеђено укупно 16 IRQ линија. Због тога је подешено да први контролер користи векторе 32–39, а други 40–47.

4.2.1.4 IRQ

Док су ISR рутине намењене обради процесорских изузетака, IRQ (Interrupt Request) рутине служе за обраду прекида које генеришу хардверски уређаји (од 32 до 47 у IDT табели). Њихово увођење било је потребно да би оперативни систем могао да реагује на догађаје који долазе од спољашњих уређаја без непрекидног проверавања њиховог стања.

Посебан значај имају прекиди тајмера и тастатуре, о којима ће бити више речи у наставку. Захваљујући IRQ рутинама, систем може да реагује на догађаје које генеришу ови уређаји чим се они догоде.

4.2.2 Драјвери

Хардвер не зна ништа о оперативном систему, већ само реагује на напон и инструкције. Да би оперативни систем могао да користи хардверске компоненте, имплементирани су одговарајући драјвери. Драјвери омогућавају комуникацију између кернела и хардвера.

4.2.2.1 Screen Driver

Након покретања кернела било је неопходно обезбедити и механизам за приказ информација на екрану. Из тог разлога имплементиран је screen драјвер који омогућава испис текста у VGA текстуалном режиму рада.

Драјвер екрана директно приступа VGA видео-меморији која се налази на адреси 0xB8000. Приликом исписа сваки карактер заузима два бајта: у једном се налази ASCII вредност, а у другом информација о боји слова и позадине. Екран је организован у 25 редова и 80 колона.

У оквиру screen драјвера имплементирани су функције за испис порука и појединачних карактера на екран, управљање курсором, брисање свих карактера на екрану и брисање последњег карактера. Поред њих, имплементирани су и помоћне функције за приказ хексадецималних вредности. Оне су коришћене приликом тестирања током развоја система.

4.2.2.2 Тајмер

Тајмер генерише хардверски прекид IRQ0 и служи оперативном систему да периодично добија сигнал од хардвера.

Иако у овом пројекту није коришћен у ту сврху, ова функционалност је додата зато што оперативни систем мора да има неки облик мерења времена. Без тајмера систем не би могао да броји протекло време или да касније реализује сложеније механизме.



Слика 4. пример рада тајмера

4.2.2.3 Тастатура

Тастатура генерише хардверски прекид IRQ1 сваки пут када корисник притисне или отпусти тастер. Приликом обраде, прекид се преводи у одговарајући карактер.

Имплементација тастатуре била је један од најважнијих корака у развоју овог пројекта, јер омогућава директну интеракцију корисника са оперативним системом.

Слика 5. пример исписа са тастатуре



4.3 Подршка за складишне уређаје

Овај део реализован је по савету ментора, са циљем да оперативни систем добије могућност трајног чувања података и комуникације са виртуелним диском.

У оквиру ове целине најпре је имплементирана подршка за PCI магистралу, која омогућава проналажење VirtIO Block уређаја. Након тога реализован је VirtIO Block драјвер, чиме је омогућено читање и упис података на виртуелни диск. На крају је додат и виши слој за рад са диском, који омогућава једноставно чување и учитавање података у оперативном систему.

4.3.1 PCI магистрала

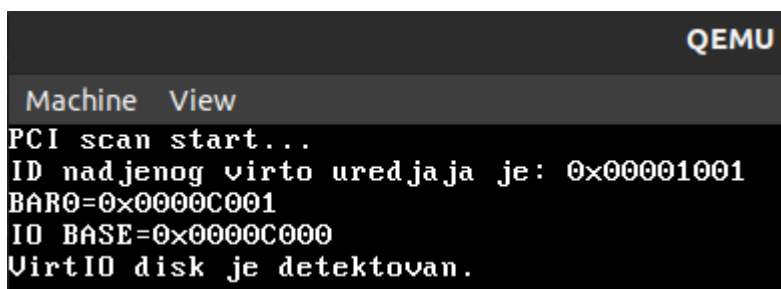
Да би оперативни систем могао да комуницира са VirtIO Block уређајем, мора прво да га пронађе. Из тог разлога имплементирана је подршка за PCI (Peripheral Component Interconnect) магистралу.

PCI магистрала омогућава комуникацију између процесора и хардверских уређаја. Сваки уређај који је повезан на PCI магистралу има конфигурациони простор у коме се налазе основне информације о уређају.

Реализовано је скенирање PCI магистрале читањем конфигурационог простора сваког уређаја. Приликом скенирања читавају се Vendor ID и Device ID вредности, на основу којих оперативни систем препознаје тип уређаја и одређује да ли је у питању VirtIO Block уређај.

Када оперативни систем пронађе VirtIO уређај, из његовог PCI конфигурационог простора читава се BAR регистар. Он садржи базну адресу преко које се касније приступа уређају. Ова адреса користи се приликом иницијализације VirtIO Block драјвера и током свих операција комуникације са виртуелним диском.

Слика 6. пример



4.3.2 VirtIO Block драјвер

Овај драјвер омогућава комуникацију са виртуелним диском. Имплементиран је како би оперативни систем могао да чита и уписује податке на диск, тако да они не постоје само привремено у RAM меморији.

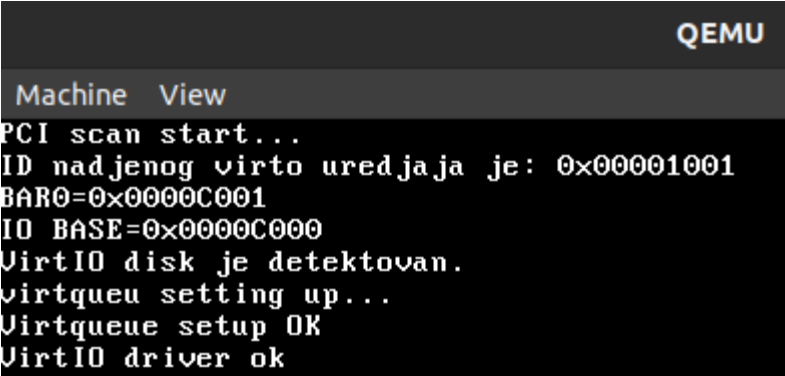
Иницијализација VirtIO Block драјвера почиње тако што прима базну I/O адресу уређаја. Та адреса је сачувана у променљивој `virtio_io_base` и користи се за приступ VirtIO регистрима током целог процеса.

Драјвер прво ресетује уређај, затим поставља статусне битове којима обавештава уређај да је препознат, да постоји драјвер и да су прихваћене подржане функционалности. Након тога бира се ред захтева, проверава се његова величина и поставља се `virtqueue` структура преко које ће се касније слати захтеви за читање и упис.

`Virtqueue` је заједничка структура у меморији коју користе и оперативни систем и VirtIO уређај. У оквиру ње налазе се `descriptor` табела, `avail ring` и `used ring`. `Descriptor`-и садрже податке о захтевима, `avail ring` служи да драјвер обавести уређај да постоји нови захтев, док `used ring` служи да уређај означи да је захтев обрађен.

Након успешног постављања `virtqueue` структуре, драјвер поставља статус `DRIVER_OK`, што значи да је VirtIO Block уређај спреман за рад. Од тада оперативни систем може да користи функције за читање и упис сектора на виртуелном диску.

Слика 7. испис након исправно имплементираног драјвера



```

QEMU
Machine View
PCI scan start...
ID nadjenog virtio uredjaja je: 0x00001001
BAR0=0x0000C001
IO BASE=0x0000C000
VirtIO disk je detektovan.
virtqueue setting up...
Virtqueue setup OK
VirtIO driver ok
  
```

4.3.3 Read и Write операција

Након иницијализације VirtIO Block драјвера имплементирани су операције читања и уписа сектора.

Read операција реализована је функцијом `virtio_blk_read_sector()` и омогућава читање једног сектора са виртуелног диска у бафер у меморији. За ову операцију формира се захтев `VIRTIO_BLK_T_IN`.

Write операција реализована је функцијом `virtio_blk_write_sector()` и омогућава упис једног сектора из меморијског бафера на виртуелни диск. За ову операцију формира се захтев `VIRTIO_BLK_T_OUT`.

Обе операције користе ланац од три `descriptor`-а. Први `descriptor` описује захтев, други садржи адресу бафера са подацима, а у трећем се налази `status byte`, преко којег се

проверава да ли је операција успешно завршена. Након припреме descriptor-a, захтев се ставља у avail ring, уређај се обавештава о захтеву преко QUEUE_NOTIFY регистра, а завршетак операције се евидентира у used ring.

4.3.4 Disk слоја

Пратећи приступ који се користи и у другим оперативним системима, уведен је диск слој, где се између кернела и конкретног драјвера уводи јединствен интерфејс за приступ диску. Због тога су имплементиране функције disk_read_sector() и disk_write_sector(), које прослеђују захтеве VirtIO Block драјверу.

На овај начин остале компоненте оперативног система не зависе од начина на који је драјвер имплементиран, што чини код једноставнијим и прегледнијим.

4.3.1 Storage

Његова улога је да омогући једноставно чување и читавање података. Док диск слој ради са секторима, storage механизам омогућава осталим компонентама оперативног система једноставно чување и читавање података.

Функција storage_save_text() уписује текст у меморијски бафер, а затим га смешта у други сектор виртуелног диска. Функција storage_load_text() чита исти сектор са диска и приказује претходно сачувани текст.

Имплементацијом storage механизма омогућено је трајно чување података, тако да ће они остати сачувани и након поновног покретања оперативног система.

4.4 Командна линија

Њена улога је да омогући унос и извршавање команди, као и тестирање реализованих функционалности.

Shell добија унос са тастатуре, обрађује га и позива одговарајуће функције. На овај начин омогућено је управљање оперативним системом без приступа појединачним компонентама.

4.4.1 Подржане команде

- `help` – приказ свих команди.
- `clear` – брисање свега на екрану.
- `about` – приказ основних информација о оперативном систему.
- `ticks` – приказ броја откуцаја системског тајмера.
- `save` – чување текста на виртуелном диску.
- `load` – читавање претходно сачуваног текста.

```
Shell pokrenut  
> help  
Komande: help, clear, ticks, about, save, load  
  
> about  
SelenaOS shell  
  
> ticks  
Timer radi.
```

Слика 8. пример коришћења функција

```
Shell pokrenut  
> save  
SAVE KOMANDA PREPOZNATA  
VirtIO write sector...  
Write OK  
Saved to disk  
  
> load  
LOAD KOMANDA PREPOZNATA  
VirtIO read sector...  
Read OK  
text: Selena grkovic
```

Слика 9. пример чувања у меморију

1. Проблеми током развоја

Током развоја пројекта наишла сам на велики број грешака. Неки проблеми били су лако отклоњиви, док су неки били много компликованији. Ти проблеми захтевали су детаљну анализу x86 архитектуре, рада са меморијом, смештања података на одговарајуће меморијске адресе и начина рада самог оперативног система. А ја сам у наставку издвојила неке од проблема са којима сам се сусрела током израде.

Први проблем током развоја се десио због прекорачења дозвољене величине boot сектора, која износи само 512 бајтова. До овог проблема дошло је јер је превише функционалности било смештено директно у bootloader. Проблем је био доста једноставан да се реши, смањила сам количину кода у bootloader-у избацавањем непотребних функционалности и поједностављењем функција.

RASPORED FIZIČKE MEMORIJE (1 MB)

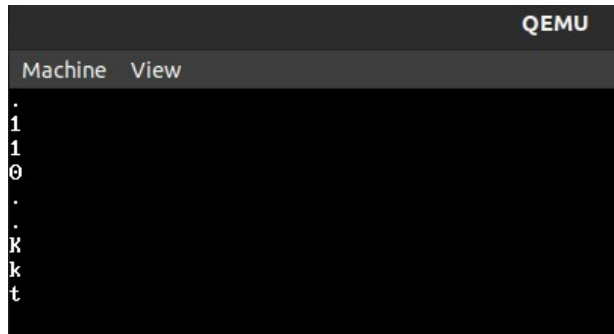
0x100000	Slobodna memorija
0xC0000	BIOS (128 KB)
0xB8000	Video memorija (32 KB)
0x90000	Stek (postavljen na 0x90000, raste nadole)
0x7C00	Boot sektor (512 B)
0x6000	Slobodna memorija
0x1000	Kernel (20 KB)
0x0400	BIOS Data Area (256 B)
0x0000	Interrupt Vector Table (IVT) (1 KB)

Од почетка константан проблем био је организација меморије. Било је потребно правилно одредити меморијске адресе на које ће бити смештени bootloader, кернел, стек и остале структуре података. Делови меморије не смеју да се преклапају, јер је доводило до неисправног рада оперативног система.

Као на пример учитавање кернела у меморију. Кернел се са диска учитавао BIOS прекидом `int 0x13` а подаци смештали преко регистара `ES:BX`. Проблем је био у томе што сам кернел учитала на једну адресу у меморији, а након преласка у `protected mode` извршавање је почињало са друге адресе. Због тога процесор извршавао погрешан садржај из меморије, што је доводило до замрзавања система. Грешку сам решила тако што сам ускладила адресу на коју BIOS учитава кернел преко `ES:BX`, linker скрипти и адресу на коју се скаче након

преласка у protected mode. Тек тад се кернел исправно извршавао. На слици сам приказала распоред првог MB моје РАМ меморије.

Следећи проблем јавио се приликом имплементације screen драјвера. На екрану су се појављивала само последња слова која су требала да се испишу, јер су се исписивала преко претходних. Упис у видео меморију је био исправан, али се текст није правилно исписивао.

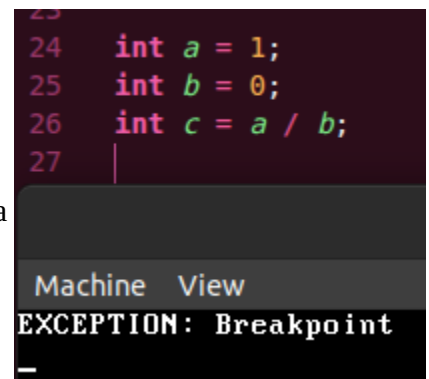


Слика 11. пример грешке

Проблем је настао јер се позиција курсора није правилно ажурирала након исписа, па су се карактери уписивали преко претходног текста. То сам решила тако што ми се након сваког исписаног карактера offset повећава за 2, а затим се нова позиција курсора шаље VGA контролеру преко портова 0x3D4 и 0x3D5.

Слика 12. пример исписа погрешне грешке

Проблем приликом имплементације ISR рутина био је што се стварни изузетак није поклапао са поруком која се исписивала на екрану. Приликом тестирања дељење са нулом очекивала сам да се активира изузетак број 0 (Division By Zero), али се уместо тога исписивала погрешна exception порука.



Испоставило се да regs->int_no показивао на погрешну вредност. Проблем је био у томе што редослед података које assembly ISR рутина поставља на стек није био усклађен са структуром registers_t, па је isr_handler() читао погрешан број прекида па из низа exception_messages[] исписивао погрешну поруку. Проблем сам решила тако што сам ускладила ISR рутине у assembly-ju, редослед података на стеку и структуру registers_t.

Следећи већи проблем појавио се приликом имплементације тастатуре, јер у почетку није реаговала како треба. PIC је слао IRQ прекиде на бројеве који су резервисани за процесорске изузетке. Због тога је било потребно ремапирати IRQ прекиде на опсег од 32 до 47.

Из претходно наведених проблема са којима сам се сусрела може се закључити да је највећи „проблем“ током израде био сам x86 асемблер, јер су ситне грешке при раду са њим биле скоро немогуће уочити. Код ручног постављања GDT-а и IDT-а, учитавања кернела или преласка у Protected Mode, један погрешан бит или адреса доводили су до замрзавања систем. Дебаговање таквог проблема сводило се на постепено додавање исписа на екран, то је чинило управо те најмање асемблерске грешке најтежим и временски најзахтевнијим за уочити.

Још један проблем се десио у додавању VirtIO block драјвера. У почетку није могао да пронађе VirtIO диск. Проблем је био у начину читања PCI конфигурационог простора.

PCI уређаји се проналазе читањем њиховог конфигурационог простора преко портова 0xCF8 и 0xCFC. У почетку читање није било исправно јер сам уместо 32-bitnog користила 16-bitno читање конфигурационог регистра, па су vendor_id и device_id имали погрешне вредности. Због тога kernel није могао да препозна VirtIO block уређај.

Проблем сам решила тако што сам додала функције за 32-bitno читање и упис I/O портова. Након тога је kernel успешно пронашао VirtIO block уређај и наставио његову иницијализацију.

Следећи проблем био је то што VirtIO уређаја није хтео да се подеси. Проблем је био јер сам у почетку BAR0 вредност користила директно, и зати је I/O base адреса била погрешна. Ову грешку сам решила тако што сам маскирала доње битове BAR0 регистра и издвојила стварну I/O base адресу. Након тога је kernel могао исправно да приступи VirtIO регистрима и настави иницијализацију уређаја.

Један од већих проблема јавио се приликом имплементације уписа и читања података са VirtIO диска. Kernel је успевао да пошаље захтев за упис, али текст који је требало да буде сачуван није могао исправно да се прочита након тога. Због тога команда за учитавање није приказивала очекивани садржај са диска.

Узрок грешке био је у подешавању flags поља код descriptor-а који показује на buffer са подацима. Код VirtIO уређаја није исто када kernel чита са диска и када уписује на диск. Код read операције уређај треба да упише податке у buffer у RAM меморији, па descriptor мора да има VRING_DESC_F_WRITE. Код write операције је обрнуто — уређај чита податке из buffer-а у RAM-у и уписује их на диск, па тај flag не сме да буде постављен.

Проблем сам решила тако што сам раздвојила подешавање descriptor-а за read и write операцију. За read сам оставила VRING_DESC_F_WRITE, јер уређај уписује податке у RAM, а за write сам уклонила тај flag, јер уређај чита податке из RAM-а. Након тога су упис и читање сектора почели исправно да раде, па је текст могао да се сачува на VirtIO диск и касније поново учита и испише.

2. Покретање система

Комплетан оперативни систем изграђује се помоћу Makefile датотеке.

- `make` – асемблира `bootloader`, компајлира `kernel`, драјвере и остале `C` модуле, повезује их помоћу `linker` скрипте и формира коначну слику диска `os-image.bin`.
- `make run` – покреће оперативни систем у QEMU емулатору и додаје `VirtIO block` уређај који се користи за тестирање читања и уписа података.

Крајњи производ изградње је датотека `os-image.bin`, која представља слику floppy диска величине 1.44 MB. Она садржи `boot` сектор и `kernel` оперативног система и користи се као диск са ког се систем покреће. Поред ње се аутоматски креира и датотека `virtio-test.img` величине 16 MB.

3. Документација

<https://angom.myweb.cs.uwindsor.ca/teaching/cs330/WritingOS.pdf>

<https://wiki.osdev.org>

<https://www.youtube.com/watch?v=-S0Z0CmwkEk>

<https://littleosbook.github.io/>

[https://www.youtube.com/watch?](https://www.youtube.com/watch?v=MwPjvJ9ulSc&list=PLm3B56ql_akNcvH8vvJRYOc7TbYhRs19M&index)

[v=MwPjvJ9ulSc&list=PLm3B56ql_akNcvH8vvJRYOc7TbYhRs19M&index](https://www.youtube.com/watch?v=MwPjvJ9ulSc&list=PLm3B56ql_akNcvH8vvJRYOc7TbYhRs19M&index)